



A Middleware for Mobile Edge Computing

A. Carrega, DITEN–University of Genoa

M. Repetto, CNIT–RU of Genoa

P. Gouvas and A. Zafeiropoulos, Ubitech Ltd.

Edge computing is the emerging paradigm that pervasively brings computing in the environment, by shifting processing from data centers to the network edge, allowing a large class of applications in growing fields, like Big Data and the Internet of Things, to be deployed in a very effective way. In this paper, we propose middleware for running applications over heterogeneous environments made of telecom networks and legacy data centers. We briefly review an emerging architecture for edge computing and

describe an integrated solution for developing and deploying modular applications in an automatic way.

Even though the cloud paradigm is widely adopted in large installations worldwide, it still falls short of supporting latency-sensitive applications that require proximity to “things” in the physical environment (devices, sensors, actuators, gateways, etc.). Augmented reality, gaming, video streaming, connected vehicles, and electrical grid protection are just a few examples of the emerging wave of Internet deployments that require mobility support and geo-distribution, in addition to location awareness and low latency.

Cloud federation may collect heterogeneous infrastructures that potentially span the entire globe; however, interaction with devices and terminals in the physical environment is still mediated by the Internet and public telecommunication networks, where topologies and typical architectures today in use introduce nonnegligible latency, jitter, and bandwidth limitations.

Instead, the concept of *fog computing* envisions “a system-level horizontal architecture that distributes computing, storage, control, and networking functions closer to the users along the cloud-to-thing continuum”; the term “fog” was just coined to recall the idea of “a cloud close to the ground”.^{1,2} The cloud-to-thing continuum is a heterogeneous environment that encompasses legacy clouds, edge installations in telecommunication access networks, and smart things. *Edge (or mist) computing* is a similar concept that only includes clouds and peripheral network installations; it exploits the rich programmable fabric (including computing, networking, and storage resources) that telecom operators are increasingly deploying in the access segment of 5G telecommunication networks.³ Edge computing provides a highly distributed, pervasive, and virtualized platform by stretching the cloud to the very edge of the network; it is not conceived as a full Cloud replacement, rather as an extension to effectively support a new breed of applications and services.

Beyond the abstraction of the underlying hardware, the big challenge will be the capability to dynamically compose, self-provision, and manage applications over a large, distributed, and heterogeneous infrastructure.^{4,5} Middleware is required between the virtualization layer and the applications that hides the complexity and heterogeneity of the underlying environment and eases the design, development, deployment, and management of distributed applications and services.

The main contribution of this paper consists in concrete design, implementation, and preliminary evaluation of edge computing. The base framework has been developed in the context of the ARCADIA project: it entails development, deployment, orchestration, and lifecycle management of distributed applications over cross-domain infrastructures, hence fitting well the structure of edge computing.⁶ This framework employs a multitier context model in the software development phase, which describes the application and its requirements, to automate orchestration, deployment, and lifecycle management.

The paper is organized as follows. The next section reviews background and related work in the field of pervasive computing. “Edge Computing Architecture” discusses an emerging architecture for edge computing, to outline specific requirements to be fulfilled. “The ARCADIA Framework” describes the overall structure of the framework to develop and deploy distributed applications, while the section “The ARCADIA Architecture and Its Usage for Edge Computing” explains why and how this framework implements the architecture for edge computing described in “Edge Computing Architecture”. “Implementation and Evaluation” discusses implementation and provides preliminary evaluation results. Finally, we give our conclusions and identify open issues and research challenges in the last section.

Background and Related Work

A growing number of applications are requesting interaction and access to smart connected devices (Internet of Things). Similarly, many factors (power consumption, size, cost) are pushing to offload complex tasks from Things to data centers.^{7,8} The Cloud paradigm permits flexible and elastic deployment of such kind of applications, but it usually virtualizes a single or a few huge data centers only. This fact definitively makes them “far” (in terms of latency) from users, physical devices, and other installations, hence making it difficult to have an effective and efficient interaction between the cloud and the physical environment.⁹

The need for closer *proximity* has pushed the design of large-scale, robust, and redundant computing systems, as *distributed* and *federated* clouds, where different data centers are jointly operated under a common interface. These models work for coarse-grained localization, e.g., by deploying a different data center for each continent, country, or region (as already done by some big cloud providers), but are not suitable for worldwide capillary coverage.

Some recent proposals already envision the possibility of building capillary clouds by exploiting telco infrastructures, for example for virtualization and offloading.^{5,7,8,10,11} In more detail, offloading is the process by which a networked device delegates part of its processing tasks, typically to the cloud. Offloading brings several benefits: increased battery life, better response time, and more processing power available. As a matter of fact, telco’s businesses are undergoing a fundamental shift, and operators are transforming their networks to meet new business challenges. In Suthar and Stolic, the authors describe the transformation of traditional wireline, wireless, voice, text, data, and web services to common compute cloud infrastructures (known as “Telco Cloud”).¹² Cloud infrastructures can be set-up on-premise, off-premise, or hybrid, based on the service level agreement, security, and maturity of services. The work in Soares and colleagues, describes how to enable telco infrastructures to adopt this new paradigm; the technologies proposed allow a telco to deploy and manage services and relative functions in a distributed cloud infrastructure.¹³

Though the broader concept of fog computing is still in its infancy, the definition of edge computing has already made important steps.^{1,4} Edge-computing architectures designed in Velasco et al. and Bruschi et al. envisage the interconnection of microinstallations at the network edge and data centers in a telco’s central office.^{10,11} The main objective here is the tight integration between service management (for placement of a virtual machine (VM)

on servers according to several policies, such as Quality of Service (QoS), energy efficiency, locality) and network management (to control connectivity among the large number of distributed installations).

All existing approaches mainly focus on virtualization aspects, but do not consider an overarching middleware that can automatically deploy and orchestrate applications over such distributed environments, including cross-domain scenarios.

Edge Computing Architecture

The growing “cloudification” wave and the need to support low-latency applications in a more effective way have motivated the interest in new cloud paradigms, which envision deployment of applications and services at the network edge.

Telco infrastructures are typically organized in access, metro, and core segments. Access segments (often referred to as “last-mile”) connect customer devices (modems, computers, phones) with the most peripheral network equipment (Broadband Remote Access Servers and similar devices), located in so-called “Points of Presence” (PoPs). Metro networks are used to interconnect several PoPs in densely populated areas (i.e., large cities), while the core network provides wide area connectivity over the entire country served by the operator.

Today, telco networks are rapidly evolving from mere communication infrastructures to rich fabrics of information and communication technologies, with computing and storage resources located close to the border (namely, in PoPs), in addition to traditional installations in central offices (see Figure 1).³ The original purpose was to replace specialized network appliances with software running on plain commercial off-the-shelf hardware (general purpose hardware largely available in the market that can be used to implement a wide range of applications and services); however, virtualization technologies also enable use of this peripheral and capillary infrastructure to provide localized cloud services, hence leading to the concept of “edge” computing (recently also referred to as “mist” computing).

The number of resources in such installations will be limited due to space, power, and cost constraints; in this respect, edge computing is seen more of an extension rather than full cloud replacement.¹⁴ Typical deployments will consist of distributed applications running core processing-intensive tasks in the cloud and lightweight tasks at the edge; the latter will provide functions with strict constraints on latency, bandwidth, and QoS.⁷

The European Telecommunications Standards Institute (ETSI) has already published an

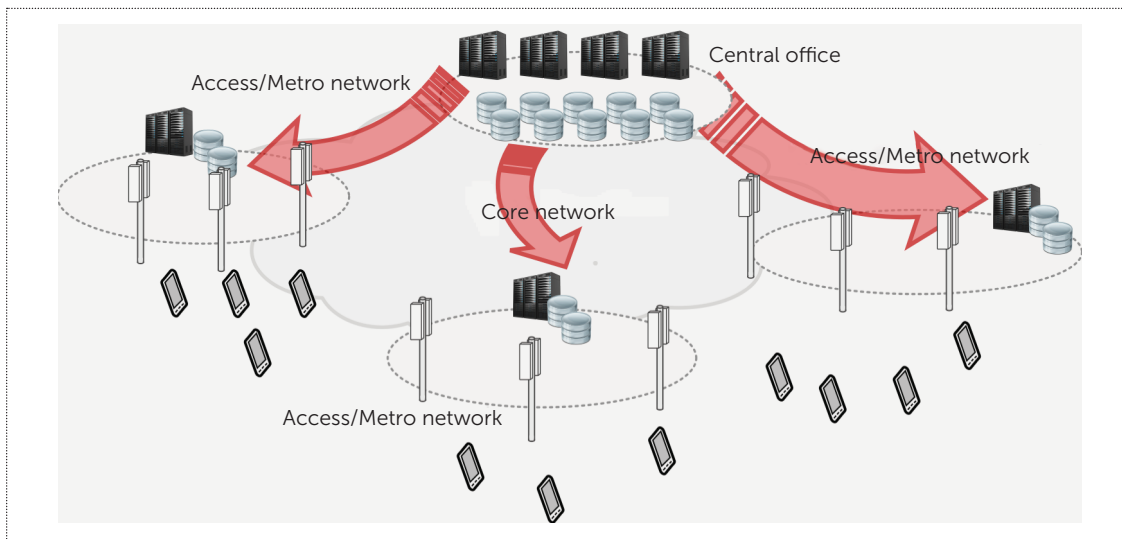


FIGURE 1. Edge computing moves computation and storage resources at the network border.

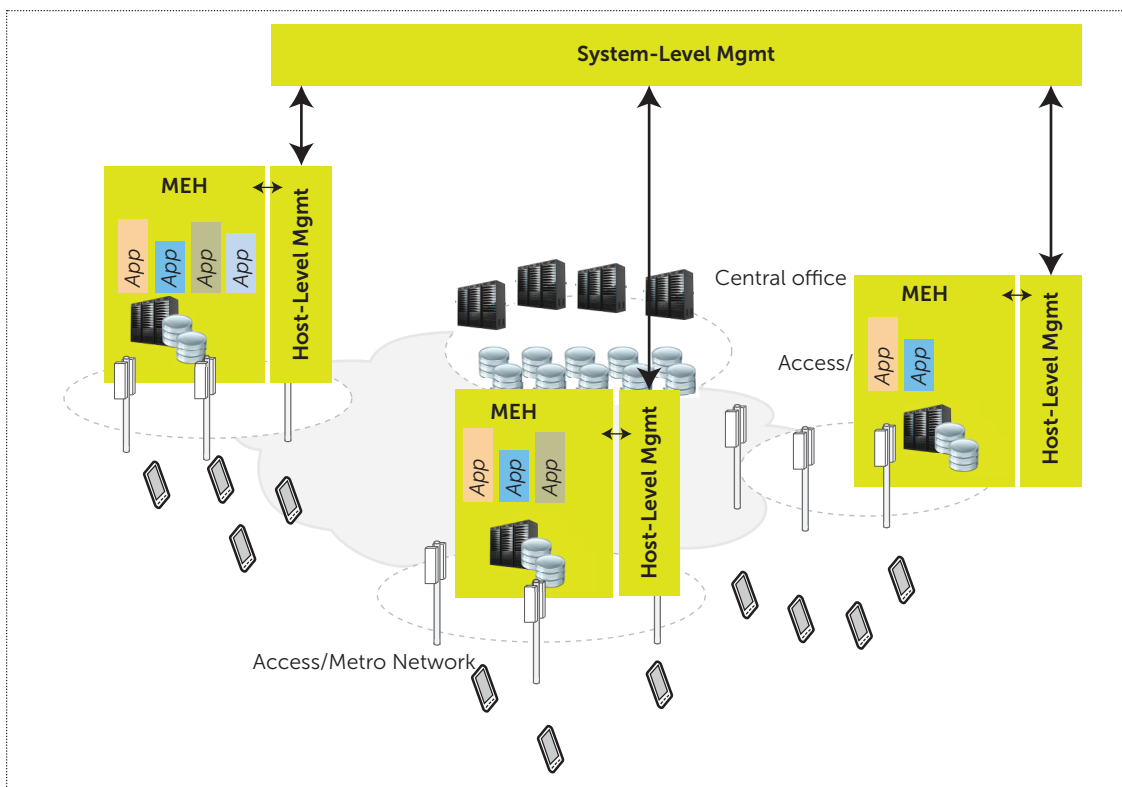


FIGURE 2. High-level view of the Mobile Edge Computing architecture.

architecture for Mobile Edge Computing (MEC), which specifically targets cellular networks.⁴ It is a reference architecture that defines the main architectural elements and their functionality; the objective is an overarching framework for distributed computing, offloading of applications

from mobile devices, and onboarding from third parties.

A high-level view of the MEC architecture is shown in Figure 2. There are two main functional layers: Host and System. The Host layer is present at each PoP to run applications, while the

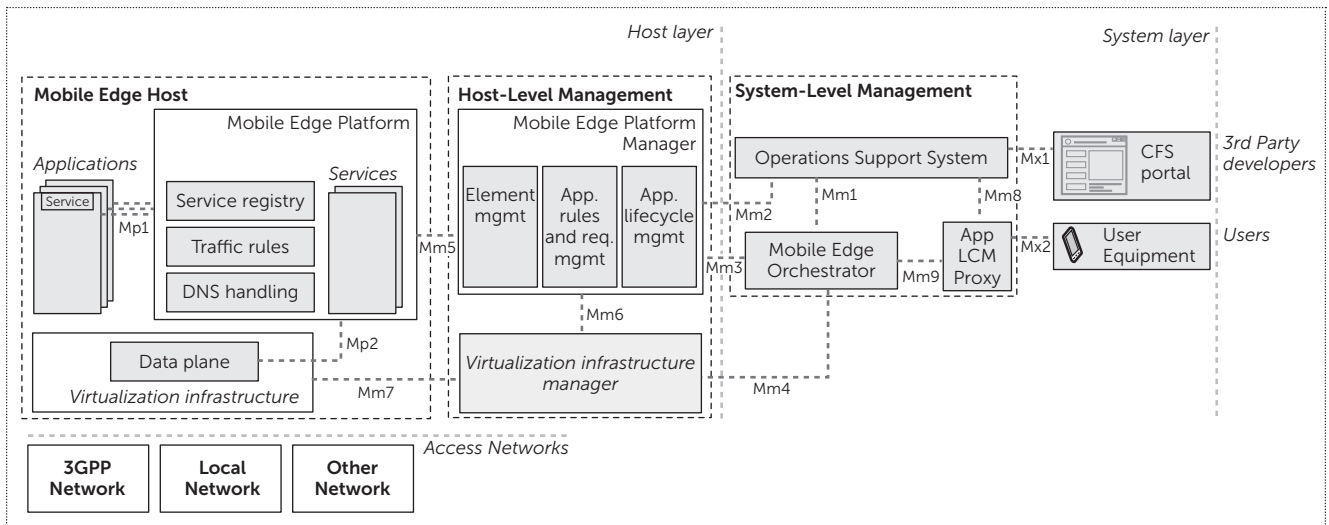


FIGURE 3. Mobile Edge Computing reference architecture.

System layer provides system-wide management functions.

The Host layer is made of a Mobile Edge Host (MEH) and its management stack (Host-Level Management). The MEH is responsible for implementing the execution environment and provides the necessary functions to run user applications (indicated as “App” in Figure 2) in an isolated sandbox. The execution environment includes both the virtualized infrastructure and ancillary services (for example, to assign Internet Protocol (IP) addresses and automatic configuration at boot, to handle Domain Name System (DNS) queries, to filter packets, etc.). The management stack controls the Host behavior; this includes provisioning of virtual resources and configuration of ancillary services. System-Level Management includes orchestration and placement of applications over the entire set of available MEHs, and interfaces for managing the whole lifecycle of user applications.

Figure 3 shows a more detailed view of each architectural block and its internal functions and components.^{4,15}

There are two main interfaces to interact with the MEC system at the system level (right side of Figure 3). The *Customer Facing Service (CFS) portal* is an entry point for providing applications. Software developers use this portal to upload their applications and to make them available to all system users; customers may select the applications they are interested in and specify when and where they are going to use them. The *User Application Lifecycle Management proxy* (User app LCM proxy, in brief) is the function responsible for instantiation,

termination, and relocation (to or from external clouds) of the applications. It enables full control of the application lifecycle directly through users’ terminals connected to the mobile network. The Operations Support System (OSS) lies in-between the external world and the operator’s network. It is widely used in a telco’s infrastructure to carry out functions like network inventory, service provisioning, network configuration, and fault management; in MEC, the OSS performs client authentication and authorization, checks software integrity, and relocates the applications between different cloud systems. Requests that are authorized by the OSS are then forwarded to the Orchestrator for further processing. The *Mobile Edge Orchestrator* is responsible for onboarding application packages, maintaining a catalog of available applications, and deploying them into MEHs according to requirements and policies (for example, constraints on latency, bandwidth, resources, and services). Further, the Orchestrator monitors the application and trigger lifecycle management actions (such as relocation, scaling, and termination).

At the Host Layer, the MEH is composed of Virtualization Infrastructure and Mobile Edge Platform, which together run user applications. User applications are software programs previously loaded in the MEC system through the CFS portal. They may run independently, they may use platform services, and they may also provide services to the platform, which can then be consumed by other applications. The *Virtualization Platform* implements the Infrastructure-as-a-Service (IaaS) model and provides virtual compute, storage, and network

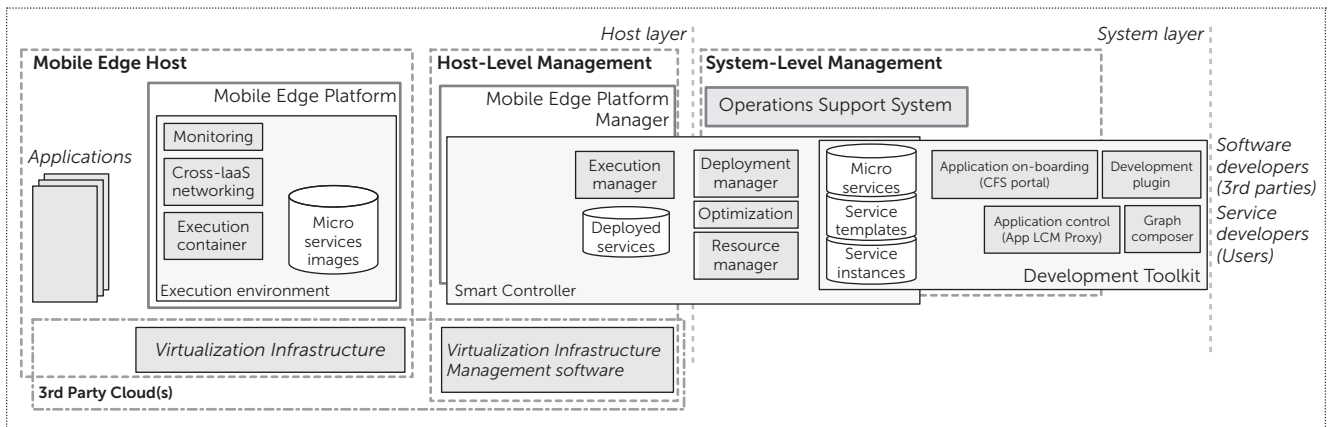


FIGURE 4. Conceptual representation of the ARCADIA framework.

resources; the data plane consists of virtual networking functions to deliver packets to applications, services, and the access networks. The *Mobile Edge Platform* (MEP) is a function that discovers, advertises, provides, and consumes services, which are provided either by users' applications or by the service provider. The MEP is also responsible for building a proper networking environment, by inspecting, classifying, firewalling, and steering packets according to deployment policies, and by setting up IP addresses and a DNS configuration.

Host-level management entails both Virtual Infrastructure management and MEP management. The *Virtualization Infrastructure Manager* provides an interface for resource provisioning; it also monitors the usage of virtual resources and reports faults and performance indications to the system-level management entities. The *MEP Manager* is split into three main functions, which are responsible for i) instantiation and termination procedures, including event reporting to the Orchestrator (App. lifecycle mgmt); ii) service authorizations, traffic rules, DNS configurations, and conflict resolution (App. rules and req. mgmt); and iii) configuration and management of logical network entities, including Broadband Remote Access Servers (terminate logical Point-to-Point links in access networks), Digital Subscriber Lines Access Multiplexers (aggregate many last-mile lines into logical flows), routers, etc. (Element Mgmt).

The ARCADIA Framework

The ARCADIA framework entails service development, deployment, and orchestration over multiple virtual heterogeneous infrastructures.¹⁶ This includes edge environments, where multiple geographically-distributed sites are used in conjunction with legacy

clouds. In this paper, we use the terms "application" and "service" interchangeably to refer to distributed software.

The main concept behind the ARCADIA approach is modeling applications in a way to facilitate automatic lifecycle management by intelligent engines. To this aim, ARCADIA applications are designed as "service graphs", which are logical software topologies where nodes represent elementary software units (*microservices*), and links represent dependencies between them. This structure identifies standalone logical components (i.e., *microservices*) that can be run in different locations, and scaled in and out according to the actual workload. The graph includes annotations that set deployment constraints, requirements, and policies from development time. Requirements and constraints can be used to provision the right set of resources to run the application, to select the proper virtualization container and operating system, to install software libraries and dependencies, and to configure *microservices* in a consistent way (usernames, passwords, IP addresses). Policies can be used to scale specific *microservices* or the entire graph according to the actual workload, to select resources from different administrative domains, to reconfigure the system in case of failures, etc. This makes applications elastic and dynamically adaptable to heterogeneous and mutable execution contexts. ARCADIA builds on and extends the Topology and Orchestration Specification for Cloud Applications standard.¹⁷

The overall framework consists of several activities, graphically depicted in Figure 4.

Microservices Development: Each *microservice* is an elementary, standalone software unit that provides a specific function/service. The executable version of the service is packaged with a control agent

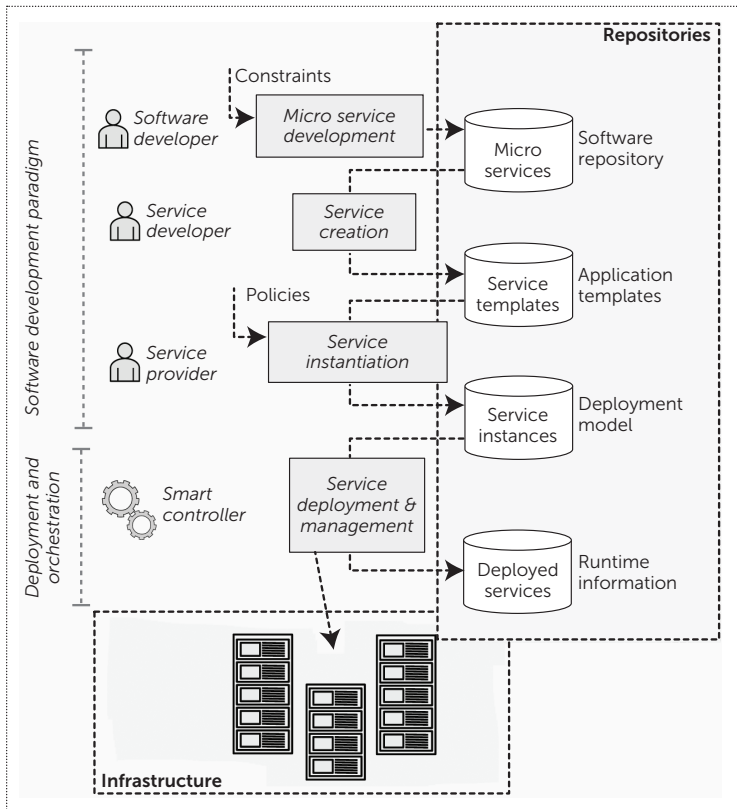


FIGURE 5. ARCADIA architecture and its mapping to ETSI MEC for mobile edge computing.

and metadata. The control agent can handle any specific lifecycle operation triggered by the system orchestrator. Metadata are organized according to a specific template, such as the Context Model, and include: component description, exposed/required functionality, deployment and execution constraints, management policies, monitoring hooks, and whatever else may be needed to support automatic deployment and orchestration by smart engines.

Service Creation: The ARCADIA Context Model envisions the concept of *programming interface*, which includes both exposed/required functionality (for instance: database backend, web server, DNS, etc.) and related configuration options (e.g., username, password, IP address). Such interface is used to create applications by combining (or *chaining*) microservices together into graph topologies (*service templates*). Two components are “linkable” in a graph if they provide complementary interfaces (e.g., MySQL client and server, Hypertext Transfer Protocol (HTTP) client and server). There is also metadata for the service graph, which contains constraints and monitoring hooks for the whole application, and a programming interface, which makes the

application reusable in other graphs in a recursive fashion.

Service Instantiation: Service graphs are then enriched with policies to create *application instances*. Policies provide a powerful mean to express optimization objectives, performance thresholds, and actions to manage the entire service lifecycle, from deployment to termination, in an automated way. They account for horizontal and vertical scaling in case the workload changes, resiliency, and migration in case of failure, redundant deployments for backup, and so on.

Service Deployment and Management: Finally, service graphs are automatically deployed by a smart orchestrator, starting from the application package that includes the graph topology, microservices definition, deployment constraints, optimization objectives, and other policies. The smart controller also manages their lifecycle; it undertakes a number of tasks:

- translates metadata and policies to optimal infrastructural configuration;
- computes the optimal placement of microservices in different execution environments;
- provisions computing, networking and storage resources;
- sets up the execution environment by booting images, installing software and libraries, and running configuration scripts;
- monitors relevant parameters and performance, analyzes measurements, and makes predictions about workload, application status, failures;
- shapes the application and its execution environment to the current conditions;
- performs management actions in case of workload changes, failures, and saturation (for example, activation, scaling, termination).

The ARCADIA Architecture and Its Usage for Edge Computing

The overall ARCADIA architecture is depicted in Figure 5, with explicit reference to the ETSI MEC framework.¹⁶ It basically consists of three main components: the interactive Development Toolkit, an intelligent orchestrator called Smart Controller, and the Execution environment.

Development Toolkit

The ARCADIA Development Toolkit enables development of microservices and service graphs through an intuitive and immediate Web User Interface (Web UI), which also includes plugins

for Integrated Development Environments (e.g., Eclipse Che; <https://eclipse.org/che>). Developers implement software as ARCADIA micro-services, including metadata for their correct instantiation and execution, and upload them into the microservices repository; telco operators are expected to provide network-related microservices as well, for example functions related to network status, mobility and QoS management. Hence, the Development Toolkit represents a superset of the functionality of the MEC CFS portal. Note that the microservices repository at this layer is mainly conceived to store their description (i.e., metadata), which is necessary to create service graphs; software images are stored in each execution environment by the Smart Controller.

Applications are created through the Graph Composer by dragging available microservices and dropping/chaining them; they are automatically saved as *service templates*. Applications may be developed by the same users (as foreseen in MEC) or by third party developers. When a user wants to activate an application, he picks the service graph from a list and annotates deployment policies and constraints (locality and mobility, QoS, security and privacy levels, response latency, redundancy and resilience level, lifecycle management, etc.), hence creating a *service instance*. The Web UI permits full control on the application lifecycle (start, stop, restart), therefore playing the same role of the App LCM Proxy in the MEC architecture.

Smart Controller

The Smart Controller splits orchestration into different functional modules. A *Deployment Manager* takes the application model instance and translates it into an optimal deployment plan by solving a placement problem that takes into consideration available infrastructures (MEHs, external clouds), their current load and performance, deployment policies, and constraints. The plan includes: i) the placement of each microservice to a specific installation, ii) additional microservices and functions to be deployed (for instance, to create virtual networking across different cloud providers), iii) the mapping of each microservice to its execution environment (VM or software container), and iv) the list of deployment actions for each microservice and the entire graph (Operating System, libraries, configurations, network topologies, etc.). The Deployment Manager also keeps an updated view of the whole system based on deployed MEHs, available resources, running applications and microservices, and topologies. An *Optimization Engine* continuously processes

monitoring information from host-level components about microservices and service graphs, and triggers reconfigurations as needed (replacement, vertical/horizontal scaling, and any other lifecycle management action set by accompanying policies), pursuing close-to-zero service disruption. The *Resource Manager* keeps an inventory of all available infrastructures: MEHs, internal, and external clouds. It maintains identity information and authentication secrets and translates internal high-level calls into platform-specific Application Programming Interfaces (APIs) to the Virtualization Infrastructure Management Software for provisioning resources, configuring the execution environment, starting, and stopping applications. The combination of these components implements all the orchestration functions envisaged by the ETSI framework.

The *Execution Manager* carries out deployment actions, including provisioning and configuration of the execution environment, retrieval of missing software images, microservices configuration, and execution of lifecycle hooks; it also collects monitoring measurements from both the infrastructure management software and specific probes available in each microservice. The Execution Manager is also responsible for setting up virtual networking, configuring access policies, and packet filtering. Run-time information is kept about active instances, their configuration, their status, and any other relevant information for lifecycle management; it can be used by the orchestrator to dynamically connect services, whenever the same microservice (like databases) could be shared among different applications. Run-time information plays a similar role of the “Service registry” in the MEC framework. Combined, the Execution Manager and the run-time repository implement the MEP Manager in the MEC architecture.

The specific software used to manage the virtualized infrastructure is not relevant for ARCADIA, since the framework is conceived to support cross-IaaS domains. The design of the Resource Manager allows to simply add drivers for different management APIs: OpenStack, Amazon Web Services, Open Cloud Computing Interface, etc.

Execution Environment

The Execution environment is deployed by the Smart Controller when the infrastructure is registered in the Resource Manager. Its main purpose is to provide common functions to all software deployed in the IaaS. For instance, it sets up virtual secure networking among all managed infrastructures. The execution environment also collects

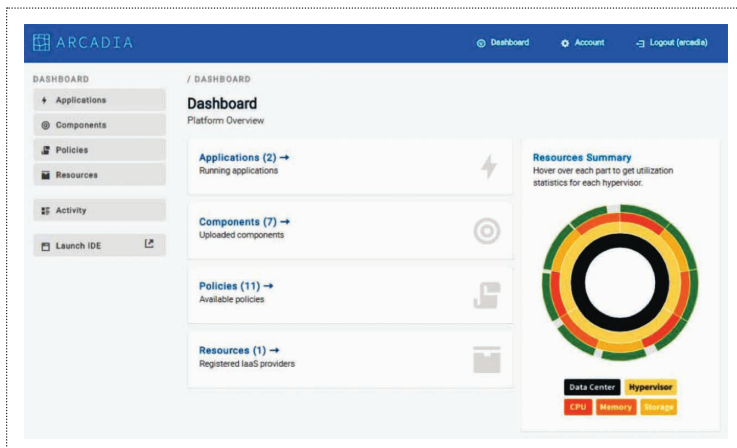


FIGURE 6. Screenshot of the ARCADIA Dashboard. The “Component” menu shows available microservices, including a brief summary of their characteristics (name, type, version, description, exposed/required interfaces, etc.).

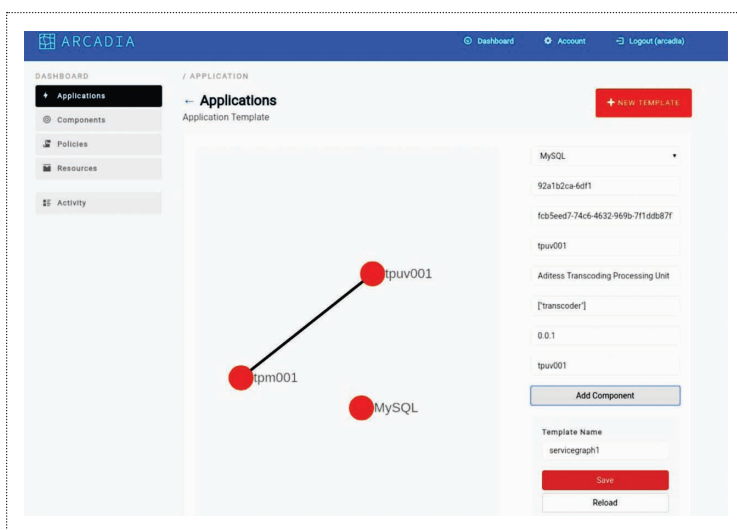


FIGURE 7. Screenshot of the ARCADIA Dashboard. The “Applications” menu allows editing of service graphs, as well as control of their lifecycle (colored boxes at bottom).

monitoring information and delivers it to the Smart Controller. This barely corresponds to the functions of the MEP for MEC.

Implementation and Evaluation

Implementation of the ARCADIA framework, its scalability, and a preliminary performance analysis are discussed in the following sections.

Implementation

The ARCADIA framework is mostly written in Java by using the Spring Boot framework (<https://spring.io>). The ARCADIA Dashboard is the entry

point that gives access to development tools and the Smart Controller (see Figure 6); the Graphical User Interface (GUI) is built with ThymeLeaf library (<http://www.thymeleaf.org>). Through the Dashboard, it is possible to upload, edit, list, and manage microservices, service graphs, policies, and infrastructures.

Microservices can be developed from scratch as *native* components or ported from existing software as *wrapped* components. The big difference lies in the architecture of their control agent. Both components are controlled by a thin-layer (agent), which is responsible for interacting with the orchestrator through a universal API. However, in the case of a native component, the agent and the component run in the same execution context (the agent is automatically injected in the class-loading process of the component “main” method), while this is not possible for a wrapped component (software binaries are used, so the agent logic and the “handling” of the agent signalling should be separated). To solve this kind of issue, the delivery of any wrapped component should be performed using container images; the Open Container Image specification (<https://github.com/opencontainers/image-spec>) is currently used to resolve any issue regarding binary diversification. Native components, which do not suffer the same limitations, may also be bundled in the form of VMs or unikernels.

The Dashboard enables simple and intuitive composition of microservices into full applications (see Figure 7); the user interface prevents logical errors during the construction of the graph. The output of the graph editor is a valid eXtensible Markup Language (XML) representation that can be inserted in the repository. The software repositories build on database technologies, like MongoDB, neo4j, and MySQL.

Deployment of a service graph implies the traversal of the directed acyclic graph to resolve first the components that have zero dependencies. After the instantiation of the “leaves”, the graph is traversed hierarchically to the top (through a recursion) until the root component is deployed. The deployment also entails a placement problem, which can be viewed as combination of two known theoretical problems, the MultiResource Generalized Quadratic Assignment Problem (MRGQAP), and the Unsplittable Flow Problem (UFP); both proved to be in the category of Nondeterministic Polynomial-time (NP)-Hard problems. Thus, an exact solution is appropriate for small instances of the problem while efficient heuristics are required to find near optimal solutions in reasonable time. OptaPlanner

(<https://www.optaplanner.org>), an open source constraint satisfaction solver, is employed to implement the optimization engine. It combines sophisticated optimization heuristics and metaheuristics (such as Tabu Search, Simulated Annealing, and Late Acceptance) with very efficient score calculation to find near-optimal solutions to NP-hard problems in short times.

Internally, policies are expressed by Drools rules-based management system (<https://www.drools.org>). Each policy consists of a set of rules; each rule is made of a condition and relative actions. Actions include lifecycle management (e.g., start, stop, destroy), run-time configurations (e.g., number of multiple incoming connections), horizontal and vertical scaling, and IaaS management. Policies and rules are edited by the Dashboard and associated to service graphs.

Conditions in policies heavily rely on the availability of run-time execution information. Monitoring is supported through a set of active and passive probes which collect information regarding availability and usage of physical resources (compute, storage, and network), resource usage per deployed microservice, and custom metrics on microservices and service graphs defined by software developers. Monitoring uses the ActiveMQ Pub/Sub framework and signaling protocol developed in Java; measurements are currently processed by a simple analysis and by a prediction engine implemented in Java, but the Kafka framework is going to be used for this purpose in next releases.

For resource management in multi-IaaS domains, the cloud abstraction library makes use of openstack4j (<http://www.openstack4j.com>). Resource management registers the whole set of infrastructure providers and provides a framework to setup the execution environment. The execution environment consists of a Linux cloud image that is used to implement the following functions: i) running wrapped component bundling; ii) build the overlay networking; iii) reporting measurements. Virtual networking is realized by Open Overlay Router (OOR; <http://www.openoverlayrouter.org>), which creates an overlay of secure connections. OOR steers traffic among the microservices, both within the same and different virtualization infrastructures; it relies on the Location/ID Separation Protocol (LISP) for the control plane and NetConf/Yang for the management plane.

Scalability

Targeting large multi-IaaS environments, scalability is of paramount importance for the ARCADIA

framework. The main design aspects that account for scalability are the following:

- support for a huge number of microservices and service graphs, by using proven database technologies to build the necessary software repositories;
- ability to deploy and manage a large number of applications by using heuristic techniques to solve the constrained placement problem;
- usage of containerization for reducing the virtualization overhead;
- virtual networking among a large number of IaaS by using the LISP protocol, which was originally designed to support IP mobility over the whole Internet.

Preliminary Performance Analysis

To evaluate the proposed framework, we implemented a simple encrypted Voice-over-IP communication service (VoIP). This application is composed of five microservices that realize the necessary functions to manage signaling, VoIP data, registration and authentication, and traversal of Network Address Translation (NAT) devices. The objective is to assess the time to automatically deploy the service in a distributed environment.

Preliminary performance analysis has shown that the framework can effectively reduce the time to develop and deploy software in multiple IaaS environments. The ability to compose application through the GUI reduces the development times from days to minutes. It has also been estimated that software maintenance time should be reduced by 30% with respect to current practice.

Real deployments of microservices took 30 seconds instead of 60 seconds, while the deployment of whole simple graphs required less than 2 minutes instead of 10 minutes. Solving the placement problem took less than 30 seconds, and scaling a service graph required approximately 10 seconds; baseline values with other approaches are about the double of these results. Though these figures are rather indicative, as the deployment time depends on the graph complexity, they demonstrate the big potential behind the proposed approach and implementation.

Conclusions

In this paper, we have outlined an effective middleware for edge environments based on the ARCADIA framework. In our opinion, the proposed solution enhances the current architecture for mobile edge computing by providing a flexible and integrated way of developing and deploying distributed applications.



We believe that further enhancements are still required at the infrastructure level to better cope with the peculiarity of mobile environments. First, better mobility support must be available to allow applications to follow users and their devices while they move around the system. As a matter of fact, OpenStack supports live migration of VMs between servers, but cross-domain migrations are still an open research issue; more investigation is needed to hand over software between different hypervisors and to effectively move data between different domains. Second, tighter control of the network should be in place beyond mere virtual overlays in order to classify and steer data packets, so to take advantage of the increasing level of programmability of telecommunication infrastructures. To tackle such kind of challenges, the INPUT project is developing Open-Volcano, a software platform specifically designed for edge computing environments, which exploits in-network programmability and provides better support for mobility and network management.^{11,18} ●●

Acknowledgments

This work was supported in part by the European Commission under the projects ARCADIA (grant no. 645372) and INPUT (grant no. 644672). The work reflects only the view of the authors, and the Commission is not liable for any use which may be made of the information contained therein.

References

1. OpenFog Consortium Architecture Working Group, "OpenFog Reference Architecture for Fog Computing," tech. paper, Feb. 2017.
2. F. Bonomi et al., "Fog Computing and Its Role in the Internet of Things," *Proc. First Ed. MCC Workshop on Mobile Cloud Computing (MCC '12)*, Helsinki, Finland, Aug. 17, 2012, pp. 13–16.
3. D. Soldani and A. Manzalini, "On the 5G Operating System for a True Digital Society," *IEEE Vehicular Technology Magazine*, vol. 10, no. 1, Mar. 2015, pp. 32–42.
4. "Mobile Edge Computing (MEC); Framework and Reference Architecture," ETSI GS MEC 003, v1.1.1, Mar. 2016; http://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/01.01.01_60/gs_MEC003v010101p.pdf.
5. "Edge Computing (MEC); Service Scenarios," ETSI GS MEC 004, v1.1.1, Nov. 2015; http://www.etsi.org/deliver/etsi_gs/MEC-IEG/001_099/004/01.01.01_60/gs_MEC-IEG004v010101p.pdf.
6. "A Novel Reconfigurable by Design Highly Distributed Applications Development Paradigm

- Over Programmable Infrastructure (ARCADIA)," ARCADIA; <http://www.arcadia-framework.eu>.
7. M. Satyanarayanan et al., "An Open Ecosystem for Mobile-Cloud Convergence," *IEEE Communications Magazine*, vol. 53, no. 3, Mar. 2015, pp. 63–70.
 8. H. Flores et al., "Mobile Code Offloading: From Concept to Practice and Beyond," *IEEE Communications Magazine*, vol. 53, no. 3, Mar. 2015, pp. 80–88.
 9. *Technology Vision 2020 Reducing Network Latency to Milliseconds*, whitepaper, Nokia Networks.
 10. L. Velasco et al., "A Service-Oriented Hybrid Access Network and Clouds Architecture," *IEEE Communications Magazine*, vol. 53, no. 4, Apr. 2015, pp. 159–165.
 11. R. Bruschi et al., "OpenVolcano: An Open-Source Software Platform for Fog Computing," *Proc. 28th Int'l Teletraffic Congress (ITC 28)*, Wuerzburg, Germany, Sep. 12–16, 2016.
 12. P. Suthar and M. Stolic, "Carrier Grade Telco-Cloud," *Proc. IEEE Asia Pacific Conf. Wireless and Mobile (APWiMob)*, Bandung, Indonesia, Aug. 27–29, 2015, pp. 101–107.
 13. J. Soares et al., "Toward a Telco Cloud Environment for Service Functions," *IEEE Communications Magazine*, vol. 53, no. 2, Feb. 2015, pp. 98–106.
 14. K. Bierzynski, A. Escobar, and M. Eberl, "Cloud, Fog and Edge: Cooperation for the Future?" *Proc. 2nd Int'l Conf. Fog and Mobile Edge Computing (FMEC 2017)*, Valencia, Spain, May 8–11, 2017.
 15. D. Sabella et al., "Mobile-Edge Computing Architecture: The Role of MEC in the Internet of Things," *IEEE Consumer Electronics Magazine*, vol. 5, no. 4, 2016, pp. 84–91.
 16. P. Gouvas et al., "A Novel Reconfigurable-by-Design Highly Distributed Applications Development Paradigm over Programmable Infrastructure," *Proc. 28th Int'l Teletraffic Congress (ITC 28)*, Wuerzburg, Germany, Sep. 12–16, 2016.
 17. "Topology and Orchestration Specification for Cloud Applications," OASIS Standard, version 1.0, Nov. 2013; <http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf>.
 18. "INPUT: In-Network Programmability for Next-Generation Personal cloud Service support," EC H2020 Programme; <http://www.input-project.eu>.

ALESSANDRO CARREGA is a software and network engineer. He is currently a post PhD research fellow at the Telecommunication Networks and

Telematics (TNT) Lab. in Genoa, Italy. TNT is a joint research laboratory between Department of Electrical, Electronic, and Telecommunications Engineering, and Naval Architecture (DITEN) at the University of Genoa, and the National Inter-University Consortium for Telecommunications (CNIT). He took part in the activities of many national and European projects (e.g., H2020 ARCADIA and INPUT, FP7 IP ECONET, PRIN EFFICIENT, FIRB GreenNet, and FIWARE) and he is an active reviewer for many different international journals and conferences (IEEE and ACM). He has coauthored several papers in international conference proceedings. In 2010, he won the best paper award at the 3rd Int'l Workshop on Green Communications (GreenCom 2010) colocated with the IEEE GLOBECOM Conference. In 2011, he was a visiting PhD scholar at Portland State University (PSU), Portland, OR, under the supervision of Prof. Suresh Singh as an active member of the FINE2 Italia – USA collaboration project. He received the BS and MS degrees in computer engineering and the PhD degree in green networking from the University of Genoa in 2005, 2007, and 2013, respectively. His main research topics are on networking (energy-aware, performance optimization, and virtualization), software routers, NFV, and SDN (in particular, OpenFlow). Finally, he is involved in collaboration with many industries, such as Telecom Italia, Broadcom, Nokia, Ericsson, Huawei, etc., and industrial fora, like GeSI. Contact him at alessandro.carrega@unige.it.

MATTEO REPETTO received the PhD degree in electronics and computer science in 2004 from the University of Genoa. From 2004 to 2009, he was a postdoc at University of Genoa. Since 2010, he is a research associate at CNIT. He has been teaching many courses in telecommunication networks and network security. He has coauthored over 40 scientific publications in international journals and conference proceedings, and he has been involved in many

different national and European research projects in the networking area. His current research interests include pervasive communications and mobility management, energy-efficient networking, software-defined networking and network function virtualization, and cloud computing. Contact him at matteo.repetto@cnit.it.

PANAGIOTIS GOUVAS received the Dipl-Ing and PhD degrees from the School of Electrical and Computer Engineering, National Technical University of Athens. He is the cofounder and R&D director of UBITECH. His research interests lay in the field of cloud computing, security, big data analytics, and autonomic networking. He has actively participated in many research and commercial projects in the aforementioned fields. He has published over 35 international papers and being member of several program committees. Contact him at pgouvas@ubitech.eu.

ANASTASIOS ZAFEIROPOULOS received the Dipl-Ing and PhD degrees from the School of Electrical and Computer Engineering, National Technical University of Athens, and the MSc degree in public policy and management from the Athens University of Economics and Business. He is currently a senior R&D architect with Ubitech Ltd. He has acquired a great deal of experience in the area of next generation networking, green IT, software defined functionalities, and at the design of context-aware, data management, and linked-data oriented solutions. He has coauthored over 40 publications in high level international journals and conferences. Contact him at azafeiropoulos@ubitech.eu.

myCS

Read your subscriptions through the myCS publications portal at <http://mycs.computer.org>.